

ساختار داده و الگوریتم

استاد : دکتر بجانی

اسفند ۱۴۰۴

عرفان باطنی

۴۰۰۱۰۰۷۹۲

پروژه ی موتور جست و جو

این پروژه با هدف پیاده‌سازی یک سامانه جستجوی مقیاس‌پذیر طراحی شده است که بر پایه مفاهیم پیشرفته درس ساختار داده شکل گرفته است. در این پروژه به جای تمرکز صرف بر مباحث تئوری، تلاش شده یک مسئله واقعی از دنیای صنعت شبیه‌سازی شود که در آن حجم داده‌ها بسیار زیاد است. در چنین شرایطی، نیاز به پردازش سریع، بهینه و کم‌هزینه به شدت احساس می‌شود. در این سناریو، ما به عنوان یک تیم فنی وظیفه داریم زیرساختی طراحی کنیم که بتواند داده‌های متنی گسترده را مدیریت کند. این داده‌ها شامل مجموعه‌ای بزرگ از متون بوده که باید به صورت کارآمد پردازش و تحلیل شوند.

در ادامه، پروژه به چند مرحله تقسیم شده که هر کدام بخشی از این سامانه را پیاده‌سازی می‌کنند. در ابتدا داده‌ها دریافت و آماده‌سازی می‌شوند، سپس داده‌های تکراری حذف شده و ساختاری مناسب برای جستجو ایجاد می‌گردد. در مراحل بعدی، داده‌ها به گونه‌ای سازمان‌دهی و فشرده می‌شوند که هم سرعت دسترسی بالا باشد و هم مصرف حافظه کاهش یابد. همچنین روش‌هایی برای مرتب‌سازی و اولویت‌بندی نتایج جستجو در نظر گرفته شده است. در نهایت نیز با استفاده از مدل‌سازی داده‌ها به صورت گراف، امکان تحلیل روابط بین آن‌ها فراهم می‌شود.

ساختار کلی این سامانه به صورت ماژولار طراحی شده و شامل بخش‌های مختلفی برای مدیریت داده، پردازش و هماهنگی بین اجزا است. در این پروژه تلاش شده از ساختار داده‌های مناسب برای هر مسئله استفاده شود تا بهترین عملکرد حاصل شود. هدف اصلی، رسیدن به تعادلی بین سرعت، مصرف حافظه و دقت نتایج است. همچنین معیارهایی مانند درستی عملکرد، کارایی و کیفیت طراحی در ارزیابی نهایی سیستم در نظر گرفته می‌شوند. در مجموع، این پروژه نشان می‌دهد که چگونه می‌توان با انتخاب درست ساختار داده و ترکیب آن‌ها، یک سیستم واقعی و کارآمد طراحی کرد.

اسپرینت اول:

من اسپرینت اول را در قالب دو فایل py انجام دادم.

در اسپرینت اول، هدف اصلی تبدیل داده‌های خام و بسیار حجیم به یک فرمت ساده و قابل پردازش بود. این داده‌ها در قالب‌هایی مانند فایل‌های متنی ساختاریافته و تو در تو قرار دارند و حجم آن‌ها به حدی زیاد است که امکان بارگذاری کامل آن‌ها در حافظه وجود ندارد. به همین دلیل، چالش اصلی این مرحله طراحی روشی برای خواندن تدریجی داده‌ها و پردازش آن‌ها بدون ایجاد مشکل در عملکرد سیستم بود. در واقع باید داده‌ها به صورت جریانی خوانده می‌شدند تا مصرف حافظه کنترل شود. خروجی مورد انتظار نیز یک فرمت سبک و خطی بود که بتواند در مراحل بعدی مورد استفاده قرار گیرد.

در این راستا، دو نوع داده مهم مورد پردازش قرار گرفتند که هر کدام نقش متفاوتی در سیستم دارند. نوع اول شامل اطلاعات ساختاری صفحات بود که ویژگی‌هایی مانند شناسه، عنوان و وضعیت صفحه را شامل می‌شود. نوع دوم شامل متن کامل مقالات بود که برای جستجو و تحلیل اهمیت دارد. هر دو نوع داده از منابع جداگانه استخراج شده و به یک قالب یکسان تبدیل شدند. این کار باعث شد داده‌ها به صورت یکپارچه و آماده برای مراحل بعدی در اختیار سیستم قرار بگیرند. در نهایت، همه اطلاعات به صورت رکوردهای جداگانه ذخیره شدند تا پردازش آن‌ها ساده‌تر شود.

در بخش مربوط به داده‌های ساختاری، چالش اصلی این بود که داده‌ها در قالبی پیچیده و غیرساده ذخیره شده بودند. برای حل این مسئله، یک روش اختصاصی برای استخراج اطلاعات مورد نیاز طراحی شد که بتواند رکوردهای مختلف را از هم جدا کند. در این روش، تنها بخش‌های مهم داده استخراج شده و از اطلاعات اضافی صرف‌نظر شد. همچنین تلاش شد داده‌ها به گونه‌ای پردازش شوند که خطاهای احتمالی در ساختار آن‌ها مدیریت شود. نتیجه این کار تولید مجموعه‌ای از داده‌های تمیز و قابل استفاده بود که شامل اطلاعات کلیدی هر صفحه می‌باشد.

در بخش مربوط به متن مقالات، مسئله اصلی حجم بسیار زیاد و ساختار تو در توی داده‌ها بود. برای مدیریت این موضوع، از روشی استفاده شد که داده‌ها را به صورت تدریجی و بخش به بخش پردازش می‌کند. در این روش، هر بخش از داده پس از پردازش از حافظه حذف می‌شود تا از افزایش مصرف حافظه جلوگیری شود.

این کار باعث شد سیستم بتواند بدون مشکل، تعداد زیادی از اسناد را پردازش کند. همچنین تنها صفحاتی که دارای متن واقعی بودند نگهداری شدند تا داده‌های غیرضروری حذف شوند.

در نهایت، خروجی هر دو بخش به یک قالب ساده و استاندارد تبدیل شد که در آن هر خط نمایانگر یک رکورد مستقل است. این ساختار باعث می‌شود که در مراحل بعدی مانند جستجو و تحلیل، داده‌ها به راحتی قابل استفاده باشند. در این اسپرینت تلاش شد تا علاوه بر استخراج صحیح داده‌ها، کارایی و پایداری سیستم نیز حفظ شود. استفاده از پردازش جریانی، کنترل حجم داده و مدیریت حافظه از مهم‌ترین نکات این مرحله بودند. این اقدامات نشان می‌دهد که سیستم طراحی شده قابلیت کار با داده‌های بزرگ را به صورت عملی دارد.

اسپرینت دوم:

در اسپرینت دوم، هدف اصلی شناسایی داده‌های تکراری در یک مجموعه بزرگ از داده‌ها بود. وجود داده‌های تکراری می‌تواند باعث افزایش حجم ذخیره‌سازی، کاهش سرعت پردازش و افت کیفیت نتایج شود. چالش مهم این بود که این کار باید روی حجم زیادی از داده‌ها و با حافظه محدود انجام شود. بنابراین مسئله فقط حذف تکراری‌ها نبود، بلکه ایجاد تعادل بین دقت، سرعت و مصرف حافظه اهمیت داشت. این موضوع یکی از مسائل واقعی در طراحی سیستم‌های بزرگ محسوب می‌شود.

برای حل این مسئله، سه روش مختلف طراحی و پیاده‌سازی شد تا بتوان عملکرد آن‌ها را به صورت عملی مقایسه کرد. در روش اول، از یک مجموعه برای نگهداری رشته‌ها استفاده شد که تمامی مقادیر یکتا را با دقت کامل ذخیره می‌کند اما مصرف حافظه بالایی دارد. در روش دوم، به جای ذخیره رشته کامل، از مقدار عددی حاصل از تبدیل رشته استفاده شد که باعث کاهش مصرف حافظه می‌شود. در روش سوم، از یک ساختار داده احتمالی استفاده شد که با مصرف بسیار کم حافظه، امکان تشخیص تقریبی تکراری‌ها را فراهم می‌کند. این تنوع در روش‌ها نشان‌دهنده بررسی دقیق گزینه‌های مختلف است.

در نهایت، با اندازه‌گیری زمان اجرا و میزان مصرف حافظه، عملکرد هر روش به صورت عملی تحلیل شد. نتایج نشان داد که روش اول دقیق و سریع است ولی حافظه بیشتری مصرف می‌کند، روش دوم تعادل مناسبی بین دقت و حافظه دارد، و روش سوم با کمترین مصرف حافظه کار می‌کند اما دقت آن تقریبی است. این تحلیل نشان می‌دهد که هیچ روش واحدی برای همه شرایط بهترین نیست و انتخاب نهایی به محدودیت‌های سیستم بستگی دارد. در این اسپرینت، تمرکز اصلی بر درک این تعادل و انتخاب آگاهانه ساختار داده مناسب بود.

اسپرینت سوم:

در این بخش از اسپرینت سوم، هدف این بود که سیستم تنها به ساخت ایندکس محدود نشود، بلکه قابلیت جستجوی واقعی نیز داشته باشد. به همین دلیل لازم بود یک موتور جستجوی ساده پیاده‌سازی شود که بتواند روی داده‌های واقعی عمل کند. علاوه بر این، یکی از اهداف مهم، مقایسه عملکرد نسخه‌های مختلف ایندکس بود. به طور خاص، باید تفاوت بین نسخه خام و نسخه فشرده از نظر سرعت و مصرف حافظه بررسی می‌شد. این موضوع نشان می‌دهد که سیستم طراحی شده فقط تئوری نیست، بلکه در عمل نیز قابل ارزیابی است.

در این راستا، یک پیاده‌سازی کامل از جستجو روی ایندکس خام انجام شد که در آن ورودی کاربر دریافت و نرمال‌سازی می‌شود و سپس مستقیماً در ساختار داده جستجو صورت می‌گیرد. این روش به دلیل سادگی، سریع‌ترین حالت ممکن را ارائه می‌دهد، زیرا هیچ پردازش اضافی مانند بازسازی داده‌ها انجام نمی‌شود. نتیجه اجرا نیز نشان داد که زمان پاسخ‌دهی تقریباً صفر است که بیانگر کارایی بسیار بالای این روش است. این موضوع به خوبی نشان می‌دهد که استفاده از ساختار داده مناسب چگونه می‌تواند سرعت جستجو را به طور چشمگیری افزایش دهد.

با این حال، اهمیت اصلی این بخش در نقش آن به عنوان یک معیار مرجع برای مقایسه است. این پیاده‌سازی نشان می‌دهد که نسخه خام اگرچه بسیار سریع است، اما مصرف حافظه بالایی دارد و در مقیاس‌های بزرگ قابل استفاده نیست. در مقابل، نسخه فشرده حافظه کمتری مصرف می‌کند ولی نیازمند پردازش اضافی است و زمان بیشتری می‌برد. بنابراین، این بخش به خوبی تعادل بین سرعت و مصرف حافظه را نشان می‌دهد. در نهایت، نتیجه‌گیری مهم این است که انتخاب روش مناسب به شرایط سیستم بستگی دارد و در عمل معمولاً از نسخه فشرده استفاده می‌شود.

برای نگهداری دیکشنری کلمات، با توجه به روشی که در پروژه استفاده کردی، استفاده از ساختار مبتنی بر نگاشت بهترین انتخاب است. چون در سیستم تو جستجو به صورت مستقیم روی کلید انجام می‌شود، این ساختار امکان دسترسی بسیار سریع را فراهم می‌کند و برای سناریوی جستجوی تک‌کلمه‌ای کاملاً مناسب است. در مقابل، ساختار درختی بیشتر برای جستجوهای پیشوندی یا کامل‌سازی کلمات کاربرد دارد که در این پروژه

نیاز اصلی نبوده است. بنابراین انتخاب تو باعث شده عملیات جستجو با بیشترین سرعت ممکن انجام شود و پیاده‌سازی نیز ساده‌تر باقی بماند.

برای فشردن‌سازی لیست شناسه مقالات، روش استفاده از فاصله بین اعداد انتخاب بسیار درستی است. چون این لیست‌ها معمولاً به صورت مرتب ذخیره می‌شوند، اختلاف بین اعداد متوالی بسیار کوچک‌تر از خود اعداد است. همین موضوع باعث می‌شود که بتوان این فاصله‌ها را با تعداد بیت کمتر ذخیره کرد و در نتیجه حجم داده به شکل قابل توجهی کاهش پیدا کند. اگر خود اعداد ذخیره شوند، فضای بیشتری مصرف می‌شود و فشردن‌سازی به خوبی انجام نمی‌شود. بنابراین استفاده از فاصله‌ها یک روش استاندارد و بهینه در سیستم‌های جستجو است.

در مورد اینکه آیا کاهش حجم ارزش هزینه بازسازی داده را دارد یا نه، پاسخ به شرایط سیستم بستگی دارد، اما در مقیاس‌های بزرگ معمولاً این کار کاملاً منطقی است. در پروژه تو مشاهده شد که نسخه خام بسیار سریع است ولی حافظه زیادی مصرف می‌کند، در حالی که نسخه فشردن حافظه کمتری نیاز دارد اما زمان بیشتری برای آماده‌سازی نتایج صرف می‌کند. در سیستم‌های واقعی که حجم داده بسیار زیاد است، محدودیت حافظه عامل تعیین‌کننده است و کاهش حجم اهمیت بیشتری پیدا می‌کند. بنابراین معمولاً هزینه زمانی بازسازی داده قابل قبول است، چون در عوض امکان کار با داده‌های بزرگ‌تر فراهم می‌شود.

اسپرینت چهارم:

در اسپرینت چهارم، هدف این بود که سیستم از یک موتور جستجوی ساده فراتر برود و بتواند به کوئری‌های تحلیلی پاسخ دهد. در این مرحله دیگر فقط جستجوی یک کلمه کافی نیست، بلکه باید روی داده‌ها عملیات‌های پیشرفته‌تری انجام شود. این شامل پرس‌وجوهایی مانند یافتن اسناد در یک بازه مشخص یا استخراج بهترین نتایج است. در واقع، سیستم به سمت یک ابزار تحلیلی واقعی حرکت می‌کند. نکته مهم این است که این قابلیت‌ها باید روی داده‌های بزرگ و واقعی به‌صورت کارآمد اجرا شوند.

در بخش اول، مسئله‌ی **Range Query** مطرح شد که هدف آن پیدا کردن داده‌ها در یک بازه مشخص است. برای این کار، سه روش مختلف پیاده‌سازی و با هم مقایسه شدند تا بهترین انتخاب مشخص شود. روش جستجوی خطی ساده بود اما برای داده‌های بزرگ بسیار کند عمل کرد. روش استفاده از آرایه مرتب‌شده و جستجوی دودویی عملکرد بسیار سریعی نشان داد و بهترین نتیجه را داشت. همچنین استفاده از درخت متوازن نیز بررسی شد، اما با وجود قدرت تئوری، در عمل کندتر و پرهزینه‌تر بود. این مقایسه نشان داد که انتخاب ساختار داده مناسب نقش کلیدی در کارایی دارد.

در ادامه، یک پیاده‌سازی عملی از بهترین روش انجام شد تا نشان داده شود که سیستم فقط در حد آزمایش نیست. داده‌ها بر اساس طول مرتب شدند و با استفاده از جستجوی دودویی، بازه مورد نظر به سرعت پیدا شد. خروجی به‌دست‌آمده نشان داد که سیستم به‌صورت واقعی و کاربردی قابل استفاده است. این بخش اهمیت تبدیل نتایج تئوری به یک سیستم عملی را نشان می‌دهد. همچنین ثابت می‌کند که می‌توان روی داده‌های واقعی با سرعت بالا پرس‌وجو انجام داد.

در بخش دوم، مسئله‌ی **Top-K** بررسی شد که هدف آن یافتن بهترین یا بزرگ‌ترین مقادیر است. برای حل این مسئله از ساختار داده هیپ استفاده شد که یکی از ابزارهای مهم در سیستم‌های واقعی است. با نگه داشتن فقط تعداد محدودی از عناصر، می‌توان بدون نیاز به مرتب‌سازی کل داده‌ها، بهترین نتایج را استخراج کرد. این روش هم از نظر زمان اجرا و هم از نظر مصرف حافظه بسیار بهینه است. استفاده از این تکنیک نشان می‌دهد که سیستم برای کار با داده‌های حجیم طراحی شده است.

در نهایت، این اسپرینت نشان داد که انتخاب ساختار داده مناسب از خود الگوریتم مهم‌تر است. مقایسه عملی بین روش‌ها کمک کرد تا بهترین گزینه بر اساس عملکرد واقعی انتخاب شود. همچنین استفاده از ابزارهایی

مانند جستجوی دودویی، درخت متوازن و هیپ نشان‌دهنده درک عمیق از مفاهیم است. مهم‌ترین نتیجه این بود که همیشه بهترین روش تئوری، بهترین انتخاب عملی نیست. بلکه باید بر اساس داده واقعی و آزمایش، تصمیم‌گیری انجام شود.

اسپرینت پنجم:

در اسپرینت پنجم، هدف این بود که داده‌های متنی به یک گراف جهت‌دار تبدیل شوند و سپس روی این گراف تحلیل انجام گیرد. ایده اصلی این است که هر صفحه به‌عنوان یک رأس و هر لینک بین صفحات به‌عنوان یک یال در نظر گرفته شود. به این ترتیب، یک گراف واقعی از ساختار وب ساخته می‌شود که رفتار لینک‌دهی بین صفحات را مدل می‌کند. این گراف پایه بسیاری از تحلیل‌های پیشرفته در موتورهای جستجو است. در واقع، سیستم از سطح متن به سطح ارتباطات بین داده‌ها ارتقا پیدا می‌کند. این مرحله یکی از مهم‌ترین گام‌ها برای درک ساختار کلان داده‌ها است.

در بخش اول، تمرکز روی ساخت گراف از داده‌ها بود که با استخراج لینک‌ها از متن صفحات انجام شد. لینک‌هایی که به شکل‌های مختلف در متن وجود داشتند، نرمال‌سازی شدند تا از ایجاد نودهای تکراری جلوگیری شود. سپس برای هر عنوان یک شناسه عددی در نظر گرفته شد تا گراف به‌صورت بهینه ذخیره شود. همچنین با حذف لینک‌های خودارجاع و استفاده از ساختار مناسب، از ایجاد یال‌های تکراری جلوگیری شد. در نهایت، گرافی با ۱۰ هزار رأس و بیش از ۶۵ هزار یال ساخته شد که نشان‌دهنده ارتباطات واقعی بین صفحات است. این خروجی نشان می‌دهد که هر صفحه به‌طور متوسط به چندین صفحه دیگر لینک دارد. نتیجه این بخش، ساخت یک گراف وب واقعی و قابل تحلیل بود.

در بخش دوم، هدف تحلیل این گراف با استفاده از مفهوم مؤلفه‌های قویاً همبند بود. برای این کار، سه الگوریتم مختلف پیاده‌سازی شد تا هم دقت و هم کارایی آن‌ها بررسی شود. دو الگوریتم اصلی یعنی تارژان و کوساراجو با پیچیدگی خطی اجرا شدند و نتایج یکسانی تولید کردند که نشان‌دهنده صحت پیاده‌سازی است. همچنین یک روش ساده‌تر برای مقایسه در نظر گرفته شد که به‌مراتب کندتر عمل کرد. نتایج نشان داد که گراف به چندین بخش تقسیم می‌شود و یک مؤلفه بسیار بزرگ در آن وجود دارد. این رفتار دقیقاً مشابه ساختار واقعی وب است که دارای یک بخش اصلی و بخش‌های پراکنده کوچک‌تر است. در نهایت، این اسپرینت نشان داد که چگونه می‌توان از داده‌های خام به تحلیل‌های سطح بالا و معنادار رسید.

عکس های کد ها و خروجی ها(به دلیل این که حجم خروجی ها زیاد است، با این وضع نت، متاسفانه نمیتوان برای شما بفرستم و چند اسکرین شات از آن ها قرار می دهم)

Local Disk (E:) > Dars > Term 9 > Data structures and algorithms			
Name	Date modified	Type	Size
Homeworks	1/4/2026 4:16 PM	File folder	
Project	4/13/2026 5:52 PM	File folder	
Slides	1/19/2026 9:46 AM	File folder	
enwiki-latest-all-titles-in-ns0.gz	2/26/2026 3:43 AM	WinRAR archive	104,614 KB
enwiki-latest-page.sql.gz	2/26/2026 3:38 AM	WinRAR archive	2,338,043 ...
enwiki-latest-pagelinks.sql.gz	2/26/2026 4:52 AM	WinRAR archive	6,699,425 ...
enwiki-latest-pages-articles-multistream1.xml-p1p41242.bz2	2/25/2026 11:51 AM	WinRAR archive	288,250 KB
enwiki-latest-pages-articles-multistream-index1.txt-p1p41242.bz2	2/25/2026 9:41 AM	WinRAR archive	222 KB

Local Disk (E:) > Dars > Term 9 > Data structures and algorithms > Project			
Name	Date modified	Type	Size
data	2/27/2026 9:31 AM	File folder	
out	2/27/2026 12:15 PM	File folder	
src	4/13/2026 5:52 PM	File folder	
Project.pdf	2/17/2026 12:58 PM	Microsoft Edge P...	4,233 KB

Local Disk (E:) > Dars > Term 9 > Data structures and algorithms > Project > src

Name	Date modified	Type	Size
 .idea	4/13/2026 7:21 PM	File folder	
 venv	2/25/2026 4:43 PM	File folder	
 main.py	2/25/2026 4:45 PM	JetBrains PyCharm...	1 KB
 sprint1_page_sql_gz.py	2/27/2026 10:08 AM	JetBrains PyCharm...	6 KB
 sprint1_parse_wiki_bz2.py	2/25/2026 4:59 PM	JetBrains PyCharm...	3 KB
 sprint2_dedup_benchmark.py	2/27/2026 10:19 AM	JetBrains PyCharm...	5 KB
 sprint3_benchmark_queries.py	2/27/2026 10:48 AM	JetBrains PyCharm...	3 KB
 sprint3_build_index.py	2/27/2026 10:40 AM	JetBrains PyCharm...	3 KB
 sprint3_compress_index.py	2/27/2026 10:47 AM	JetBrains PyCharm...	4 KB
 sprint3_query_compressed.py	2/27/2026 10:48 AM	JetBrains PyCharm...	2 KB
 sprint3_query_index.py	2/27/2026 10:39 AM	JetBrains PyCharm...	1 KB
 sprint4_range_compare.py	2/27/2026 12:02 PM	JetBrains PyCharm...	6 KB
 sprint4_range_query.py	2/27/2026 11:58 AM	JetBrains PyCharm...	1 KB
 sprint4_topk.py	2/27/2026 11:57 AM	JetBrains PyCharm...	1 KB
 sprint5_build_graph_from_text.py	2/27/2026 12:15 PM	JetBrains PyCharm...	3 KB
 sprint5_scc_compare.py	2/27/2026 12:18 PM	JetBrains PyCharm...	6 KB

Local Disk (E:) > Dars > Term 9 > Data structures and algorithms > Project > out

Name	Date modified	Type	Size
 compressed_lexicon.pkl	4/13/2026 7:31 PM	PKL File	29,196 KB
 compressed_postings.bin	4/13/2026 7:31 PM	BIN File	16,676 KB
 graph_10k.pkl	4/13/2026 7:42 PM	PKL File	255 KB
 inverted_index.pkl	4/13/2026 7:30 PM	PKL File	59,234 KB
 page_meta.jsonl	4/13/2026 7:21 PM	JSONL File	657 KB
 wiki_docs.jsonl	4/13/2026 7:25 PM	JSONL File	1,041,801 ...

```
E:\Dars\Term 9\Data structures and algorithms\Project\out\wiki_docs.jsonl - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
wiki_docs.jsonl
1 {"id": 10, "title": "AccessibleComputing", "text": "#REDIRECT [[Computer accessibility]]\n\n{{Rcat shell|{{R
2 {"id": 12, "title": "Anarchism", "text": "{{Short description|Political philosophy and movement}}\n\n{{Other use
3 {"id": 13, "title": "AfghanistanHistory", "text": "#REDIRECT [[History of Afghanistan]]\n\n{{Redirect category
4 {"id": 14, "title": "AfghanistanGeography", "text": "#REDIRECT [[Geography of Afghanistan]]\n\n{{Redirect cate
5 {"id": 15, "title": "AfghanistanPeople", "text": "#REDIRECT [[Demographics of Afghanistan]]\n\n{{Redirect cate
6 {"id": 18, "title": "AfghanistanCommunications", "text": "#REDIRECT [[Communications in Afghanistan]]\n\n{{Red
7 {"id": 19, "title": "AfghanistanTransportations", "text": "#REDIRECT [[Transport in Afghanistan]]\n\n{{Redirect
8 {"id": 20, "title": "AfghanistanMilitary", "text": "#REDIRECT [[Afghan Armed Forces]]\n\n{{Redirect category s
9 {"id": 21, "title": "AfghanistanTransnationalIssues", "text": "#REDIRECT [[Foreign relations of Afghanistan]]\
10 {"id": 23, "title": "AssistiveTechnology", "text": "#REDIRECT [[Assistive technology]]\n\n{{Redirect category
11 {"id": 24, "title": "AmoeboidTaxa", "text": "#REDIRECT [[Amoeba]]\n\n{{Redirect category shell|1=\n\n{{R from Ca
12 {"id": 25, "title": "Autism spectrum", "text": "#REDIRECT [[Autism]]\n\n{{Redirect category shell|{{R from a
13 {"id": 27, "title": "AlbaniaHistory", "text": "#REDIRECT [[History of Albania]]\n\n{{Redirect category shell|1
14 {"id": 29, "title": "AlbaniaPeople", "text": "#REDIRECT [[Demographics of Albania]]\n\n{{Redirect category she
15 {"id": 30, "title": "AsWeMayThink", "text": "#REDIRECT [[As_We_May_Think]]\n\n{{Redirect category shell|1=\n {
16 {"id": 35, "title": "AlbaniaGovernment", "text": "#REDIRECT [[Politics of Albania]]\n\n{{Redirect category she
17 {"id": 36, "title": "AlbaniaEconomy", "text": "#REDIRECT [[Economy of Albania]]\n\n{{Redirect category shell|1
18 {"id": 39, "title": "Albedo", "text": "{{Short description|Ratio of how much light is reflected back from a bc
19 {"id": 40, "title": "AfroAsiaticLanguages", "text": "#REDIRECT [[Afroasiatic languages]]\n\n{{Redirect categor
20 {"id": 42, "title": "Artificallanguages", "text": "#REDIRECT [[Constructed language]]\n\n{{Rcat shell|{{R fr
21 {"id": 46, "title": "Abacus", "text": "#REDIRECT [[Abacus]]\n\n{{Redirect category shell|1=\n\n{{R from CamelCas
22 {"id": 47, "title": "Abalone", "text": "#REDIRECT [[Abalone]]\n\n{{Redirect category shell|1=\n\n{{R from CamelC
23 {"id": 48, "title": "'Abbādid", "text": "#REDIRECT [[Abbadid dynasty]]"}
24 {"id": 49, "title": "Abbess", "text": "#REDIRECT [[Abbess]]\n\n{{Redirect category shell|1=\n\n{{R from CamelCas
```

```
E:\Dars\Term 9\Data structures and algorithms\Project\out\page_meta.jsonl - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
page_meta.jsonl
1 {"id": 10, "ns": 0, "title": "AccessibleComputing", "redirect": 0}
2 {"id": 12, "ns": 0, "title": "Anarchism", "redirect": 0}
3 {"id": 13, "ns": 0, "title": "AfghanistanHistory", "redirect": 0}
4 {"id": 14, "ns": 0, "title": "AfghanistanGeography", "redirect": 0}
5 {"id": 15, "ns": 0, "title": "AfghanistanPeople", "redirect": 0}
6 {"id": 18, "ns": 0, "title": "AfghanistanCommunications", "redirect": 0}
7 {"id": 19, "ns": 0, "title": "AfghanistanTransportations", "redirect": 0}
8 {"id": 20, "ns": 0, "title": "AfghanistanMilitary", "redirect": 0}
9 {"id": 21, "ns": 0, "title": "AfghanistanTransnationalIssues", "redirect": 0}
10 {"id": 23, "ns": 0, "title": "AssistiveTechnology", "redirect": 0}
11 {"id": 24, "ns": 0, "title": "AmoeboiTaxa", "redirect": 0}
12 {"id": 25, "ns": 0, "title": "Autism_spectrum", "redirect": 0}
13 {"id": 27, "ns": 0, "title": "AlbaniaHistory", "redirect": 0}
14 {"id": 29, "ns": 0, "title": "AlbaniaPeople", "redirect": 0}
15 {"id": 30, "ns": 0, "title": "AsWeMayThink", "redirect": 0}
16 {"id": 35, "ns": 0, "title": "AlbaniaGovernment", "redirect": 0}
17 {"id": 36, "ns": 0, "title": "AlbaniaEconomy", "redirect": 0}
18 {"id": 39, "ns": 0, "title": "Albedo", "redirect": 0}
19 {"id": 40, "ns": 0, "title": "AfroAsiaticLanguages", "redirect": 0}
20 {"id": 42, "ns": 0, "title": "ArtificialLanguages", "redirect": 0}
21 {"id": 46, "ns": 0, "title": "Abacus", "redirect": 0}
22 {"id": 47, "ns": 0, "title": "Abalone", "redirect": 0}
23 {"id": 48, "ns": 0, "title": "Abbadid", "redirect": 0}
24 {"id": 49, "ns": 0, "title": "Abbes", "redirect": 0}
25 {"id": 50, "ns": 0, "title": "AbbevilleFrance", "redirect": 0}
```